

Organização e Arquitetura de Computadores I

(OAC I) - ACH2034 - EP II

Alan Moura Silva
T94
15436668
alan.silva02@usp.br

Derick dos Santos Arriado
T94
15636042
derick.arriado@usp.br

Isabella Cremonesi Morija
T94
14579951
isa.cmorija@usp.br

Karina Yang Chen
T94
15466658
ka_yangchen@usp.br

Resumo—Este relatório referente ao segundo Exercício Programa de OAC I apresenta a implementação de dois algoritmos de ordenação — Bubble Sort e Quick Sort — utilizando a linguagem Assembly. O exercício tem como foco a comparação de desempenho entre os algoritmos por meio de testes práticos, cuja análise é apresentada através de um gráfico ilustrativo.

I. PROPOSTA

Este trabalho tem como objetivo implementar os algoritmos de ordenação Bubble Sort e Quick Sort para ordenar um vetor de números.

Serão aplicados os principais conceitos da linguagem Assembly abordados em sala de aula, como o uso de registradores, instruções, leitura de arquivos, manipulação de loops e vetores. A execução e os testes dos programas serão realizados no simulador MARS, voltado para a arquitetura MIPS.

Para facilitar o desenvolvimento, os algoritmos serão inicialmente implementados em uma linguagem de alto nível (C) e, em seguida, traduzidos para Assembly MIPS.

II. CÓDIGOS DE REFERÊNCIA EM C

Conforme especificado pelo professor, desenvolvemos um código em C como referência para a montagem da mesma execução em Assembly MIPS, incluso na submissão do trabalho.

III. CONJUNTO DE DADOS PARA A ANÁLISE

Para a avaliação e comparação dos dois algoritmos de ordenação, foram utilizados arquivos .txt contendo números randômicos do tipo float de 10, 100, 1000 (fornecido pelo professor) e 10000 entradas. Desse modo, pode-se

Os três arquivos que foram gerados pelo grupo foram feitos utilizando um script em Python com especificações notadas do arquivo fornecido:

- Intervalo numérico: Números entre 1.00000 e 9.99999

(em módulo).

- Precisão: 5 casas decimais.
- Aleatoriedade: Distribuição uniforme de valores; Definição posterior de sinal positivo/negativo em mesma probabilidade.
- Seed fixa (42) para resultados reproduzíveis. *"A resposta para a vida, o universo e tudo mais"- O Guia do Mochileiro das Galáxias (Douglas Adams).*

```
import random

def generate_random_numbers(n):
    numbers = []
    for _ in range(n):
        # Gera numero positivo entre 1 (inclusive) e 10 (exclusive)
        num = random.uniform(1, 10 - 1e-9)
        # ultima parte evita arredondamento para 10.0

        # sinal positivo ou negativo
        if random.choice([True, False]):
            num = -num

        # Formata para 5 casas decimais
        numbers.append(f"{num:.5f}")
    return numbers

# Configuracao
random.seed(42)
sizes = [100, 1000, 10000]

# Gera arquivos
for size in sizes:
    data = generate_random_numbers(size)
    with open(f"{size}numeros.txt", "w", encoding="utf-8") as f:
        f.write("\n".join(data))
```

Listing 1: Geração de números de entrada para os códigos de algoritmos de ordenação.

IV. FLUXOS DO PROGRAMA

Foram feitos fluxogramas para melhor compreensão do funcionamento do fluxo de chamada e retorno de funções (e rótulos) do programa, cujo funcionamento é detalhado a seguir.

1) Bubble Sort

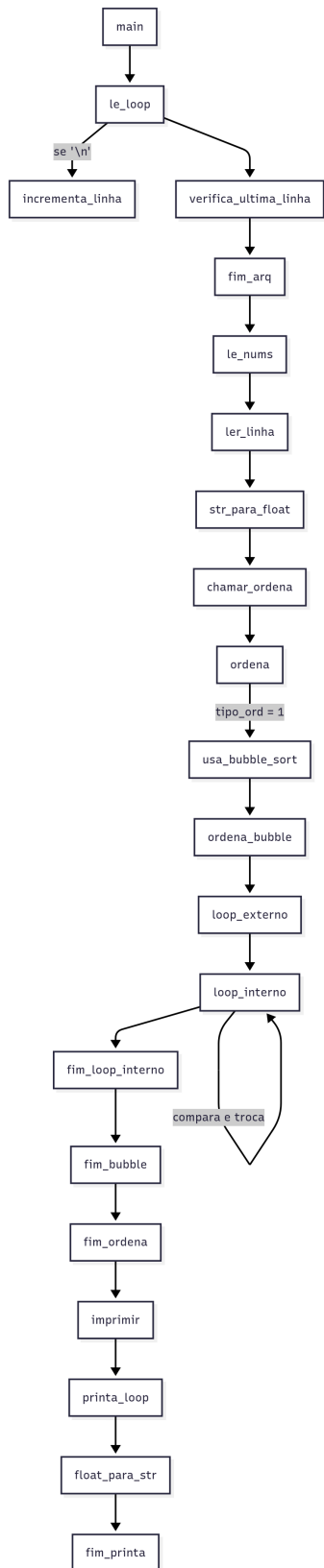


Figura 1: Fluxograma de branches do programa em Bubble Sort.

2) Quick Sort

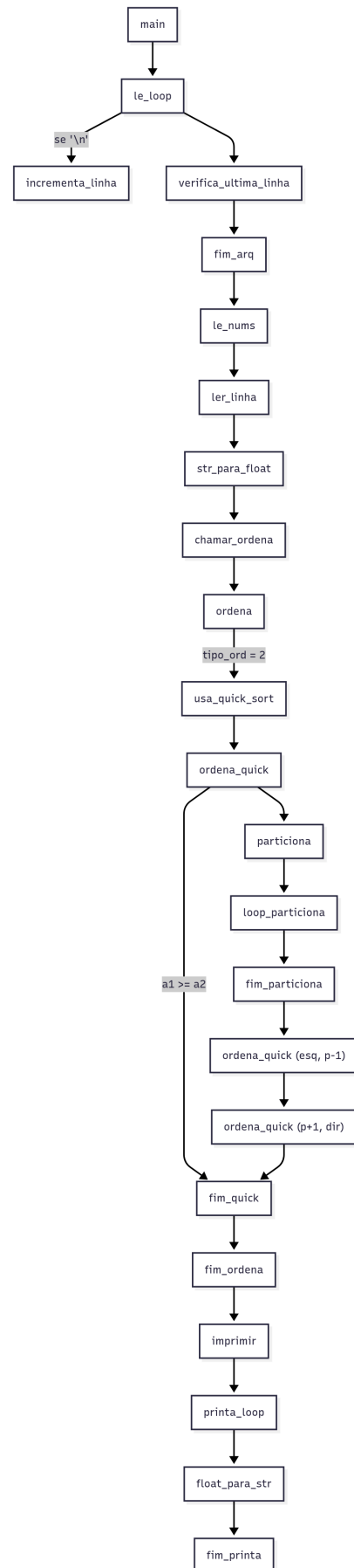


Figura 2: Fluxograma de branches do programa em Quick Sort.

V. IMPLEMENTAÇÕES DOS ALGORITMOS DE ORDENAÇÃO

Esta seção detalha o processo da implementação da estrutura principal do programa e as particularidades dos algoritmos Bubble Sort e Quick Sort.

A. Seção de Dados (.data)

- `arq_nome`: String com o caminho completo do arquivo a ser processado.
- `msg_ord`: Mensagem "ELEMENTOS ORDENADOS" que será escrita no arquivo de saída para identificar a seção dos resultados.
- `newline`: Caractere de nova linha ("\n") usado para formatação da saída.
- `byte_lido`: Espaço de 1 byte para armazenar cada caractere lido individualmente do arquivo durante o processo de parsing.
- `linha_buf`: Buffer de 32 bytes para armazenar uma linha completa antes da conversão para float.
- `flt_zero`: Valor 0.0 usado em comparações, inicializações e como base para operações de soma sucessiva.
- `flt_dec`: Valor 100000.0 utilizado na conversão de float para string para preservar 5 casas decimais.
- `flt_um`: Valor 1.0 usado nas operações de soma sucessiva que substituem as instruções `cvt.s.w`.
- `tipo_ord`: Define o algoritmo de ordenação a ser utilizado (1 = Bubble Sort, 2 = Quick Sort), permitindo flexibilidade na escolha do método

```
.data
arq_nome:      .asciiz "C:\Users\alanm\
                OneDrive\ rea de Trabalho\Nova pasta\
                EP-OAC\EP-OAC\epOac\Arquivos_de_numero
                \2000numeros.txt"
msg_ord:       .asciiz "\n\nELEMENTOS
                ORDENADOS\n\n"
newline:       .asciiz "\n"

byte_lido:     .space 1
linha_buf:     .space 32

flt_zero:      .float 0.0
flt_dec:       .float 100000.0
flt_um:        .float 1.0

tipo_ord:      .word 2 # 1 para Bubble Sort
                e 2 para QuickSort
```

Listing 2: Trecho do código.

B. Função Principal (main)

A função `main` coordena todo o fluxo de execução do programa, desde a abertura do arquivo até a gravação dos resultados ordenados. Esta função implementa um pipeline completo de processamento: primeiro realiza uma passada pelo arquivo apenas para contar as linhas (necessário para alocação de memória), depois realiza uma segunda passada para

efetivamente ler e processar os dados, executa a ordenação e finalmente grava os resultados formatados.

1) Abertura e Contagem de Linhas

- 1) **Abertura do arquivo**: Utiliza `syscall 13 (open)` para abrir o arquivo em modo leitura (flag 0), armazenando o file descriptor em `$s0`
- 2) **Loop de contagem**: O loop `le_loop` lê byte por byte usando `syscall 14 (read)`, incrementando um contador (`$s1`) a cada '\n' encontrado
- 3) **Tratamento de EOF**: Verifica se a última linha possui conteúdo mesmo sem terminador '\n', garantindo contagem correta
- 4) **Fechamento**: Usa `syscall 16 (close)` para liberar o file descriptor após a contagem

2) Alocação de Memória

- **Cálculo do tamanho**: Multiplica o número de linhas por 4 bytes (tamanho de um float em MIPS)
- **Alocação dinâmica**: Utiliza `syscall 9 (sbrk)` para alocar memória no heap, armazenando o endereço base em `$s2`

3) Leitura dos Números

- **Reabertura do arquivo**: Necessária pois o ponteiro está no final após a contagem
- **Loop de leitura**: Para cada linha chama `ler_linha` para capturar a string completa
- **Conversão**: Utiliza `str_para_float` para converter cada string em valor de ponto flutuante
- **Armazenamento**: Calcula o endereço correto no array e armazena usando `s.s (store single)`

4) Ordenação

- **Chamada da função**: Invoca `ordena` passando tamanho do array (`$a0`), tipo de ordenação (`$a1`) e ponteiro para o array (`$a2`)
- **Flexibilidade**: O tipo de ordenação é lido da variável `tipo_ord`, permitindo mudança de algoritmo

5) Saída dos Resultados

- **Abertura para escrita**: Reabre o arquivo com flag 9 (`write + create + truncate`)
- **Cabeçalho**: Escreve a mensagem identificadora usando `syscall 15 (write)`
- **Loop de saída**: Para cada número do array ordenado, converte para string e escreve seguido de '\n'
- **Finalização**: Fecha o arquivo e termina o programa com `syscall 10 (exit)`

```
# loop que conta as linhas do arquivo
le_loop:
    li $v0, 14
    add $a0, $s0, $zero
    la $a1, byte_lido
    li $a2, 1
    syscall
```

```

slt $t0, $v0, $zero
bne $t0, $zero, fim_arq # vai para o '
fim_arq'
beq $v0, $zero, verifica_ultima_linha #
vai para o verifica_ultima_linha

lb $t0, byte_lido
li $t1, 10 # utilizado para o new line, '\
n' equivalente em C
beq $t0, $t1, incrementa_linha

# se n o for new line, marca que h
conte do na linha, e ai vai ao '
le_loop'
li $s7, 1
j le_loop

(...)

```

Listing 3: Trecho do código.

C. Função de Ordenação (ordena)

A função ordena é responsável por selecionar o método de ordenação a ser utilizado. Sua assinatura segue a orientação do professor, recebendo como parâmetros o tipo de ordenação, o tamanho do vetor e o ponteiro para o vetor de dados.

Propósito: Função dispatcher que escolhe o algoritmo de ordenação baseado no parâmetro `tipo_ord` e gerencia adequadamente a pilha e registradores.

a) Parâmetros

- `$a0`: Tamanho do array (número de elementos)
- `$a1`: Tipo de ordenação (1 = Bubble Sort, 2 = Quick Sort)
- `$a2`: Ponteiro para o array de floats

b) Funcionamento

- **Preservação de contexto:** Salva `$ra`, `$s0`, `$s1` e `$s2` na pilha para manter integridade dos dados
- **Cópia de parâmetros:** Transfere argumentos para registradores `$s` para preservá-los durante chamadas de função
- **Seleção de algoritmo:** Usa comparações sequenciais para determinar qual função chamar
- **Chamada específica:** Configura parâmetros apropriados para cada algoritmo (Bubble Sort usa tamanho, Quick Sort usa índices)
- **Retorno:** Restaura contexto da pilha e retorna ponteiro do array ordenado em `$v0`

```

# aqui come a a fun o ordena, e vai ser
chamados parametros que a gente havia
especificado anteriormente
# '$s0' o tamanho, '$s2' o tipo (
especificado em .data) e '$s2' a array
ordena:
addi $sp, $sp, -16
sw $ra, 12($sp)
sw $s0, 8($sp)

```

```

sw $s1, 4($sp)
sw $s2, 0($sp)

add $s0, $a0, $zero
add $s1, $a1, $zero
add $s2, $a2, $zero

li $t0, 1
beq $s1, $t0, usa_bubble_sort

li $t0, 2
beq $s1, $t0, usa_quick_sort

j usa_bubble_sort

(...)

```

Listing 4: Trecho do código.

D. Algoritmo Bubble Sort

O Bubble Sort é um algoritmo de ordenação simples e intuitivo que funciona comparando elementos adjacentes e trocando-os se estiverem na ordem errada. Embora tenha complexidade $O(n^2)$ e não seja eficiente para grandes datasets, sua implementação é direta e educativa. Este algoritmo é chamado "bubble" porque os elementos maiores "borbulham" para o final do array a cada passada, similar a bolhas subindo na água.

1) Função ordena_bubble

Propósito: Implementa o algoritmo Bubble Sort para ordenação crescente usando comparações de ponto flutuante e trocas in-place.

a) Parâmetros

- `$a0`: Ponteiro para o array de floats
- `$a1`: Tamanho do array (número de elementos)

b) Funcionamento Detalhado

Estrutura de Loops Aninhados:

- **Loop externo:** Controla o número de passadas necessárias (n passadas para n elementos)
- **Loop interno:** Realiza comparações adjacentes em cada passada
- **Otimização:** Reduz o alcance do loop interno a cada passada, pois os maiores elementos já estão posicionados

Loop Externo (loop_externo):

- **Contador de passadas:** Variável `$t0` inicia em 0 e vai até o tamanho do array
- **Condição de parada:** Quando todas as passadas foram realizadas, o array está completamente ordenado
- **Garantia:** Cada passada garante que pelo menos um elemento (o maior da subsequência não ordenada) atinge sua posição final

Loop Interno (loop_interno):

- **Comparações adjacentes:** Varre elementos de 0 até (tamanho - passada_atual - 1)
- **Cálculo de endereços:** Usa multiplicação por 4 e soma de base para acessar array[j] e array[j+1]
- **Otimização de alcance:** Diminui o número de comparações a cada passada pois elementos finais já estão ordenados

Comparação e Troca:

- **Carregamento:** Usa l.s (load single) para carregar valores em registradores de ponto flutuante (\$f0 e \$f1)
- **Comparação:** Utiliza c.le.s (compare less or equal single) para verificar se array[j] ≤ array[j+1]
- **Decisão de troca:** Branch condicional bc1t pula a troca se elementos estão na ordem correta
- **Execução da troca:** Armazena valores nas posições opostas usando s.s (store single)

c) Características de Desempenho

- **Complexidade temporal:** $O(n^2)$ no pior caso, $O(n)$ no melhor caso (array já ordenado)
- **Complexidade espacial:** $O(1)$ - ordenação in-place
- **Estabilidade:** Mantém ordem relativa de elementos iguais

```
# ----- Algoritmo Bubble Sort
# ----- #
ordena_bubble:
    addi $sp, $sp, -16
    sw $ra, 12($sp)
    sw $s0, 8($sp)
    sw $s1, 4($sp)
    sw $s2, 0($sp)
    add $s0, $a0, $zero
    add $s1, $a1, $zero
    li $t0, 0

    (...)
```

Listing 5: Trecho do código.

E. Algoritmo Quick Sort

O Quick Sort é um algoritmo de ordenação divide-e-conquista altamente eficiente que funciona selecionando um elemento como pivô e particionando o array de modo que elementos menores fiquem à esquerda e maiores à direita do pivô. Sua implementação recursiva e estratégia de divisão resultam em complexidade média $O(n \log n)$, tornando-o significativamente mais rápido que algoritmos quadráticos para grandes datasets. A escolha do último elemento como pivô simplifica a implementação, embora outras estratégias possam otimizar casos específicos.

1) Função ordena_quick

Propósito: Implementa o algoritmo Quick Sort recursivo utilizando estratégia de particionamento com pivô fixo (último elemento).

2) Parâmetros

- \$a0: Ponteiro para o array de floats
- \$a1: Índice inicial (low) do subarray a ser ordenado
- \$a2: Índice final (high) do subarray a ser ordenado

a) Funcionamento Recursivo

Condição de Parada:

- **Verificação de base:** Testa se low < high usando slt (set less than)
- **Caso base:** Se low ≥ high, o subarray tem 0 ou 1 elemento, já está ordenado
- **Eficiência:** Evita processamento desnecessário de subarrays triviais

Processo de Divisão:

- **Particionamento:** Chama função particiona que reorganiza elementos em torno do pivô
- **Retorno do pivô:** Recebe índice da posição final do pivô após particionamento
- **Garantia:** Após particionamento, pivô está em sua posição final no array ordenado

Chamadas Recursivas:

- **Subarray esquerdo:** Chama ordena_quick(array, low, pivot-1) para elementos menores que o pivô
- **Subarray direito:** Chama ordena_quick(array, pivot+1, high) para elementos maiores que o pivô
- **Convergência:** Recursão termina quando todos os subarrays são triviais (tamanho ≤ 1)

Gerenciamento de Pilha:

- **Preservação:** Salva 6 registradores (\$ra, \$s0-\$s4) devido à natureza recursiva
- **Restauração:** Garante integridade do contexto ao retornar de cada chamada recursiva

3) Função particiona

Propósito: Reorganiza o array colocando elementos menores ou iguais ao pivô à esquerda e elementos maiores à direita, retornando a posição final do pivô.

a) Estratégia de Particionamento

- **Escolha do pivô:** Utiliza sempre o último elemento do subarray como pivô (estratégia simples mas eficaz)
- **Esquema de dois ponteiros:** Mantém ponteiro i para próxima posição de elemento menor e ponteiro j para varredura
- **Invariante:** Elementos de low até i são menores ou iguais ao pivô

b) Processo de Particionamento

Inicialização:

- **Carregamento do pivô:** Calcula endereço do último elemento e carrega em \$f0

- **Ponteiros iniciais:** Define $i = \text{low} - 1$ (posição do último elemento menor) e $j = \text{low}$ (posição de varredura)
- **Preparação:** Configura registradores para o loop principal

Loop Principal (loop_particiona):

- **Varredura completa:** Percorre de low até high-1, examinando cada elemento
- **Comparação:** Para cada $\text{array}[j]$, compara com pivô usando c.le.s
- **Condição de troca:** Se $\text{array}[j] \leq \text{pivô}$, incrementa i e troca $\text{array}[i]$ com $\text{array}[j]$
- **Manutenção de invariante:** Garante que elementos à esquerda de $i+1$ são menores ou iguais ao pivô

Posicionamento Final do Pivô:

- **Troca final:** Move pivô da posição high para posição $i+1$ (primeira posição de elemento maior)
- **Resultado:** Pivô fica entre elementos menores (esquerda) e maiores (direita)
- **Retorno:** Índice $i+1$ indica a posição final do pivô no array

c) Características de Eficiência

- **Complexidade:** $O(n)$ para cada chamada de particiona
- **In-place:** Não requer memória adicional além de variáveis locais
- **Determinística:** Sempre produz o mesmo particionamento para mesma entrada

```
# ----- Algoritmo Quick Sort
# ----- #
ordena_quick:
    addi $sp, $sp, -24
    sw $ra, 20($sp)
    sw $s0, 16($sp)
    sw $s1, 12($sp)
    sw $s2, 8($sp)
    sw $s3, 4($sp)
    sw $s4, 0($sp)

    add $s0, $a0, $zero
    add $s1, $a1, $zero
    add $s2, $a2, $zero

    slt $t0, $s1, $s2
    beq $t0, $zero, fim_quick

    add $a0, $s0, $zero
    add $a1, $s1, $zero
    add $a2, $s2, $zero
    jal particiona
    add $s3, $v0, $zero

    # a metade do lado esquerdo
    add $a0, $s0, $zero
    add $a1, $s1, $zero
    addi $a2, $s3, -1
    jal ordena_quick
```

```
# a metade do lado direito
add $a0, $s0, $zero
addi $a1, $s3, 1
add $a2, $s2, $zero
jal ordena_quick

(...)
```

Listing 6: Trecho do código.

F. Funções Auxiliares

As funções auxiliares são componentes essenciais que fornecem funcionalidades de baixo nível para operações de entrada/saída e conversão de dados. Estas funções encapsulam operações complexas e repetitivas, promovendo reutilização de código e abstração de detalhes de implementação. São especialmente importantes neste contexto pois implementam conversões manuais entre tipos de dados, contornando limitações ou preferências de implementação específicas do ambiente MIPS.

1) Função ler_linha

Propósito: Lê uma linha completa do arquivo caractere por caractere, construindo uma string terminada em null no buffer fornecido.

Esta função implementa um parser de linha robusto que trata adequadamente diferentes cenários de fim de arquivo e previne overflow de buffer. É fundamental para o processamento de arquivos texto, especialmente quando o formato exato das linhas pode variar ou quando se deseja controle preciso sobre a leitura.

a) Funcionamento

- **Leitura sequencial:** Utiliza `syscall 14 (read)` em loop para ler um byte por vez
- **Detecção de nova linha:** Para quando encontra caractere `'\n'` (ASCII 10)
- **Construção de string:** Acumula caracteres válidos no buffer `linha_buf`
- **Terminação:** Adiciona byte nulo ao final para formar string C válida
- **Retorno de tamanho:** Fornece comprimento da linha lida para validação

b) Tratamento de Casos Especiais

- **EOF com conteúdo:** Se arquivo termina sem `'\n'` mas há caracteres lidos, considera como linha válida
- **EOF sem conteúdo:** Retorna 0 indicando fim real do arquivo
- **Proteção de buffer:** Limita leitura a 30 caracteres para prevenir overflow
- **Erro de leitura:** Detecta e trata erros de I/O apropriadamente

```
ler_linha:
```

```

    la $t0, linha_buf
    li $t1, 0

# ler caractere
le_char:
    li $v0, 14
    add $a0, $s0, $zero
    la $a1, byte_lido
    li $a2, 1
    syscall

# verifica o para verificar se houve
# erro na leitura
slt $t2, $v0, $zero
bne $t2, $zero, fim_linha

# aqui para veirficar se chegou ao final
# do arquivo
beq $v0, $zero, verifica_conteudo

lb $t2, byte_lido
li $t3, 10 # new line
beq $t2, $t3, fim_linha

#armazena o caractere lido durante o loop
add $t4, $t0, $t1
sb $t2, 0($t4)
addi $t1, $t1, 1

# verifica o se n o      maior que o
# tamanho do 'buffer'
li $t5, 30
bne $t1, $t5, le_char

j fim_linha

(...)

```

Listing 7: Trecho do código.

2) Função str_para_float

Propósito: Converte uma string numérica (formato decimal) para valor em ponto flutuante IEEE 754, implementando parsing manual completo.

Esta função é particularmente importante pois implementa conversão de string para float sem depender de instruções automáticas de conversão do processador. Isto pode ser necessário em implementações específicas, ambientes simulados ou para fins educacionais, demonstrando compreensão profunda da representação numérica.

a) Processo de Conversão

Análise Lexical:

- **Deteção de sinal:** Verifica primeiro caractere para determinar se número é negativo
- **Separação sintática:** Identifica posição do ponto decimal para separar partes inteira e fracionária
- **Validação:** Processa apenas caracteres numéricos válidos (0-9 e '.')

Parsing da Parte Inteira:

- **Acumulação decimal:** Multiplica valor acumulado por 10 e soma novo dígito
- **Conversão ASCII:** Subtrai 48 ('0') para converter caractere em valor numérico
- **Construção progressiva:** Constrói valor inteiro dígito por dígito da esquerda para direita

Parsing da Parte Decimal:

- **Contagem de casas:** Mantém contador de dígitos após ponto decimal
- **Acumulação:** Constrói valor decimal como inteiro temporário
- **Cálculo do divisor:** Determina potência de 10 apropriada para normalização

Conversão para Float:

- **Método de soma sucessiva:** Converte inteiros para float adicionando 1.0 repetidamente (substitui cvt.s.w)
- **Normalização decimal:** Divide parte decimal pelo divisor apropriado usando div.s
- **Composição final:** Soma parte inteira convertida com parte decimal normalizada
- **Aplicação de sinal:** Multiplica por -1 se número original era negativo

Particularidade Técnica: Evita completamente o uso de instruções cvt.s.w e cvt.w.s, demonstrando implementação manual de conversão de tipos.

```

str_para_float:
    add $t0, $a0, $zero
    li $t1, 0 # parte inteira
    li $t2, 0 # parte decimal
    li $t3, 0 # divisor
    li $t4, 1 # sinal
    li $t5, 0 # flag
    lb $t6, 0($t0)
    li $t7, 45 # '-'
    bne $t6, $t7, conv_digitos
    li $t4, -1
    addi $t0, $t0, 1

    (...)

```

Listing 8: Trecho do código.

3) Função float_para_str

Propósito: Converte um valor em ponto flutuante para sua representação string formatada, implementando serialização manual completa.

Esta função complementa str_para_float e é essencial para a saída formatada dos resultados. A implementação manual permite controle preciso sobre formato, precisão e tratamento de casos especiais, além de evitar dependências de bibliotecas de conversão padrão.

a) Processo de Conversão

Preparação e Normalização:

- **Análise de sinal:** Usa `c.lt.s` para determinar se valor é negativo comparando com 0.0
- **Tratamento de negativos:** Adiciona '-' ao buffer e converte para valor absoluto
- **Configuração de precisão:** Multiplica por 100000.0 para trabalhar com 5 casas decimais

Método de Truncamento Inovador:

- **Multiplicação para preservar decimais:** Valor $\times$ 100000 move parte decimal para posições inteiras
- **Truncamento por subtração:** Usa loop de subtração sucessiva de 1.0 em vez de instruções de truncamento
- **Separação de componentes:** Divide resultado por 100000 para obter partes inteira e decimal

Formatação da Parte Inteira:

- **Extração de dígitos:** Usa divisão e módulo por 10 para extrair dígitos individuais
- **Conversão ASCII:** Adiciona 48 para converter valor numérico em caractere
- **Tratamento de zero:** Caso especial quando parte inteira é zero
- **Inversão necessária:** Dígitos são gerados em ordem reversa, requer inversão do buffer

Formatação da Parte Decimal:

- **Otimização de zeros:** Remove zeros à direita desnecessários antes da formatação
- **Adição de ponto:** Insere '.' apenas se há dígitos decimais significativos
- **Cálculo de divisor:** Determina potência de 10 apropriada baseada no número de dígitos
- **Extração ordenada:** Gera dígitos decimais da esquerda para direita

b) Características Avançadas

- **Controle de precisão:** Limita saída a máximo 5 casas decimais
- **Otimização de formato:** Remove zeros trailing para representação mais limpa
- **Robustez:** Trata adequadamente valores zero, negativos e casos extremos
- **Terminação adequada:** Garante string terminada em null para uso posterior

```
float_para_str:
    addi $sp, $sp, -32
    sw $ra, 28($sp)
    sw $t0, 24($sp)
    sw $t1, 20($sp)
    sw $t2, 16($sp)
    sw $t3, 12($sp)
    sw $t4, 8($sp)
    sw $t5, 4($sp)
    sw $t6, 0($sp)
```

```
# verifica se negativo
l.s $f0, flt_zero
c.lt.s $f12, $f0
b.c1f positivo
li $t0, 45 # '-'
sb $t0, 0($a0)
addi $a0, $a0, 1
sub.s $f12, $f0, $f12

(...)
```

Listing 9: Trecho do código.

VI. ANÁLISE COMPARATIVA DE DESEMPENHO

Esta seção apresenta a análise do desempenho utilizando o tempo de execução dos algoritmos Bubble Sort e Quick Sort.

Utilizamos uma abordagem na qual executa o arquivo do programa por meio do PowerShell, com o comando Measure-Command:

```
PS C:\Users\isacm> Measure-Command { java -jar .\Mars.jar nc .\ep0ac\ep.asm }
```

Figura 3: Modo de execução do programa que conta o tempo de execução.

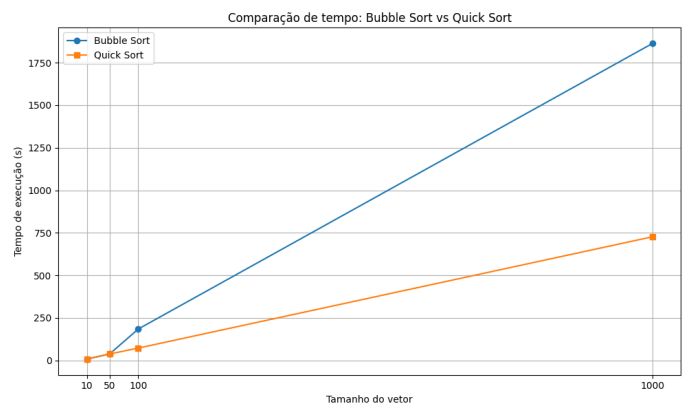


Figura 4: Gráfico comparando desempenho dos algoritmos (utilizamos 10, 50, 100 e 1000 entrada).

A comparação mostra que, para vetores pequenos, o **tempo de execução** dos algoritmos é semelhante. No entanto, **conforme o tamanho cresce, o Quick Sort apresenta uma vantagem cada vez maior**. Para 1000 elementos, o **Bubble Sort** levou cerca de 1863 segundos, enquanto o **Quick Sort** executou em 726 segundos.

Portanto, o Quick Sort se mostra claramente mais eficiente em termos de tempo, especialmente para entradas maiores.

VII. DESAFIOS ENFRENTADOS

As dificuldades encontradas durante o desenvolvimento do projeto foram significativas. A implementação não está otimizada, buscando não utilizar funções de conversão, o que criou gargalos lógicos claros e que podem ser observados pelos tempos do gráfico quando comparados com os tempos de execução apresentados em linguagens como C. A lógica desenvolvida certamente não é a mais eficiente possível.

Inicialmente, tentamos mensurar o desempenho dos algoritmos através do contador de instruções nativo do MIPS. Contudo, a supervisão da ferramenta tornava a execução excessivamente lenta, inviabilizando esta abordagem. O código MIPS desenvolvido já possui performance naturalmente baixa, como já pontuado, e as ferramentas de monitoramento do MARS agravam ainda mais essa lentidão, tornando impraticável a contagem precisa de instruções. Optamos então por medir o tempo de execução usando PowerShell com Measure-Command, mesmo sendo uma métrica menos robusta.

VIII. CONSIDERAÇÕES FINAIS

Este relatório detalhou a implementação e a análise comparativa dos algoritmos de ordenação Bubble Sort e Quick Sort em Assembly MIPS. O projeto proporcionou uma valiosa oportunidade de aplicar conceitos fundamentais da arquitetura de computadores e da programação de baixo nível.

A. Uso de IA como ferramenta de apoio

Este projeto contou com o suporte de ferramentas de IA para otimizar algumas tarefas. Foram utilizados ChatGPT, DeepSeek e Claude Sonnet 4, que auxiliaram na:

- Elaboração deste Relatório: apoio na redação, estruturação e revisão do texto.
- Obtenção de insights para desenvolver o código.
- Depuração de bugs (Debug) no programa principal.
- Geração dos fluxogramas em código MMD.
- Geração dos gráficos de desempenho dos algoritmos em Python (Matplotlib).